KU LEUVEN

FACULTEIT INGENIEURSWETENSCHAPPEN

Warps, thread divergence, and striding
Lab 3
Geavanceerde computerarchitectuur

Wouter Fouquet
Florian Vandenbogaerde
Master elektronica ICT
Academiejaar 2024-2025

CONTENTS

Exercise **1**
0.1 Part 1 - warps . . . 1
0.2 Part 2 - thread divergence . . . 1
0.3 Pointers part 1 . . . 1
0.4 Pointers part 2 . . . 2

1 Methodology **3**
1.1 Warps . . . 3
1.2 Thread starvation . . . 3
1.3 Striding . . . 3
1.4 Thread divergence . . . 4
1.5 Results . . . 4
1.5.1 No striding . . . 4
1.5.2 Thread starvation . . . 4
1.5.3 Striding . . . 5
1.5.4 Thread Divergence . . . 6

Conslusion **9**

EXERCISE

0.1 Part 1 - warps

- Implement a kernel that inverts an image (calculate x = 255 - x, where x is an element of (0, 255), for each color component - RGB).
- Assess how the number of threads (full/ partial warps) influences the computation time.
- See pointers below!

0.2 Part 2 - thread divergence

- Augment the inversion kernel, so that the R color component is calculated as 1D-Gaussian blur of its neighbors. Namely, the ith red pixel r_i is obtained as:

$$r_i = 0.1r_{i-2} + 0.25r_{i-1} + 0.5r_i + 0.25r_{i+1} + 0.1r_{i+2}$$

  i.e., a Gaussian weighted sum of its 1D-neighboring red pixel values.
- Note that the green and blue values are still processed as before (x = 255 - x)
- Assess how thread divergence impacts the computation time.
- See pointers below!

0.3 Pointers part 1

- Implement the inverse kernel and use the provided image load/ save function (Toledo - Docs) to invert an image. You can use as many threads as ther are input elements.
- Implement striding in the above kernel - if difficult, first implement striding in a simpler kernel that handles a short 1D sequence, for example, using 12 threads, 16 elements (0,1,...,15), and adding 1 to each element.
- Check that the inversion kernel, with the added striding, generates a fully-inverted image even when using less threads than there are input data elements. For example, consider a ratio of 1:;16 (threads:elements).
- Invoke the inverse kernel from a loop, where you progressively use more threads. Time the kernel on every iteration (see cheat sheet).
- Get the average execution time, as the time may vary. For example, due to preemption by the OS.

- Note that on some setups, the GPU may take more time to finish when first invoked - you can avoid this by calling the function once before actually timing it.

0.4 Pointers part 2

- Make a second kernel, where the R color component is processed according to the provided equation.
- Check that the returned image is indeed different.
- Now make a second kernel, which instead of processing RGBRGB...RGB operates on an input format of RR...RGGG...GBB...B (re-format the data correspondingly).
- Evaluate how the execution time changes with/ without thread divergence.

1.1 Warps

A warp consists of 32 threads that execute together. These threads perform the same instruction simultaneously but operate on different data. To calculate the number of warps, we can use the following formula:

$$\text{Number of warps} = \left(\frac{\text{Threads per block}}{32}\right) \times \text{Number of blocks}$$

This calculation helps efficiently organize processing within a GPU.

1.2 Thread starvation

Thread starvation occurs when there are not enough threads available to process all elements in a workload, leading to unprocessed data or inefficient execution. If the total number of pixels (M×N×C) is greater than the available threads, and no striding is used, many pixels will remain unprocessed. This results in incomplete image computations. To mitigate thread starvation, techniques like striding or increasing the number of blocks and threads can be used to ensure full coverage of the image.

1.3 Striding

Striding is a technique to efficiently distribute work across available threads when processing large datasets. Instead of assigning a single thread to each data element, striding allows threads to process multiple elements by skipping a fixed number of indices in memory. This ensures that all elements are covered even when there are fewer threads than data points.When the number of available threads is smaller than the total number of pixels (M×N×C), each thread can handle multiple pixels using striding. This maximizes GPU utilization and avoids unprocessed pixels.

1.4 Thread divergence

A section of a warp may have greater complexity, leading to a longer computation time. As a result, the remaining parts of the warp become idle, temporarily ceasing to contribute to data processing while still occupying a core. To address this issue, we begin by splitting the incoming array in the code based on the R, G, and B values. As observed in the task, the computational cost is higher for the R values compared to G and B. To optimize GPU core usage and prevent thread divergence, we introduce a second kernel to process these values separately.

1.5 Results

1.5.1 No striding

Table 1.1 presents the timing results obtained without applying striding. We make sure that there will always be enough threads available to compute the entire image in parallel.

Number of threads per block	Number of warps per block	Blocks per grid	Timing [ms]
1	1	3686400	19,37
2	1	1843201	16,36
4	1	921601	14,81
8	1	460801	14,24
16	1	230401	13,99
32	1	115201	13,90
64	2	57601	12,11
128	4	28801	9,96
256	8	14401	14,88
512	16	7201	31,05
1024	32	3601	31,05

Table 1.1: Table without striding

1.5.2 Thread starvation

If we use fewer threads than the total number of input elements (960×1280×3) and do not apply striding, only a portion of the image will be processed and inverted (see Figure 1.1). In this case, with a 1:16 ratio, many elements remain unprocessed.

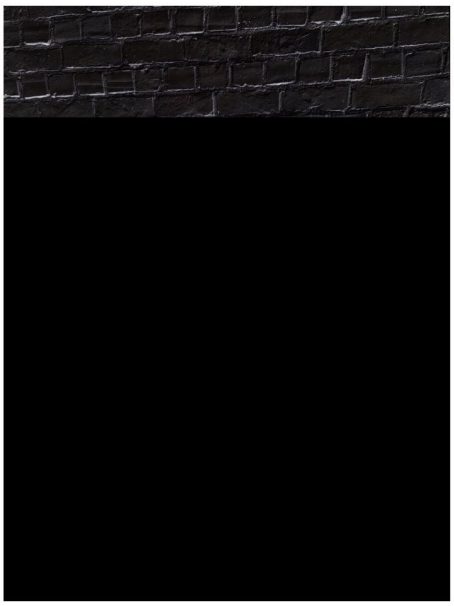


Figure 1.1: Output without enough threads and striding

1.5.3 Striding

By implementing striding, we can reuse threads when there are more elements to process than available threads. The measurement results obtained using striding are shown in Table 1.2. We use a single block per grid and double the number of threads at each step to clearly observe the differences in execution time.

Number of threads per block	Number of warps per block	Number of Blocks	Timing [ms]
1	1	1	483,63
2	1	1	248,67
4	1	1	130,66
8	1	1	72,67
16	1	1	43,81
32	1	1	27,88
64	2	1	13,96
128	4	1	7,06
256	8	1	3,61
512	16	1	1,92
1024	32	1	1,04

Table 1.2: Table with striding

We expect that using a multiple of 32 threads per block will yield the highest performance, as this aligns with the warp length. If there are fewer than 32 threads, partial warps occur, resulting in suboptimal thread utilization. As seen in the measurement results, when too few threads are used (causing striding to occur), increasing the number of threads speeds up the process by enabling greater parallelism. The orginal and inverted images are shown in figure 1.2 and 1.3.



Figure 1.2: Orginal image

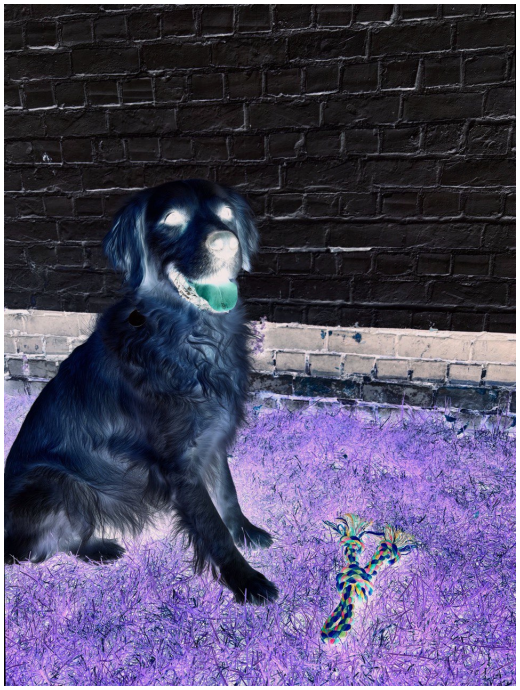


Figure 1.3: Inverted Image

The code for this section can be seen in code snippet 1.1.

1.5.4 Thread Divergence

In this section, we will explore the impact of thread divergence on GPU performance by using the red pixels to describe a Gaussian formula, such as

$$r_i = 0.1r_{i-2} + 0.25r_{i-1} + 0.5r_i + 0.25r_{i+1} + 0.1r_{i+2}$$

i.e., a Gaussian weighted sum of its 1D neighboring red pixel values. As a result, the thread handling the red component will take longer to compute than the threads responsible for processing blue or green components. The green and blue pixels are simply inverted, so the processing of these pixels takes much less time than that of the red pixels, leading to thread divergence.

1.5.4.1 Interleaved bitstream (RGBRGBRGB...)

In the case of an interleaved bitstream (RGBRGBRGB...), we expect significant thread divergence because, within the same warp, both red, blue, and green pixels need to be processed. The processing time for red pixels is longer due to the Gaussian formula, while the blue and green pixels are processed much faster. As a result, this disparity in processing times within the same warp leads to thread divergence, as some threads will be idle while others are still processing the more complex red pixels.

The measurements with the interleaved bitstream are shown in Table 1.3. These measurements were taken again with a single block and a variable number of threads, which will result in striding.

Number of threads per block	Number of warps per block	Number of Blocks	Timing [ms]
1	1	1	580,61
2	1	1	296,68
4	1	1	154,60
8	1	1	83,93
16	1	1	44,52
32	1	1	23,28
64	2	1	14,06
128	4	1	7,12
256	8	1	4,07
512	16	1	2,38
1024	32	1	1,37

Table 1.3: Measurements with interleaved bitstream

1.5.4.2 Planar bitstream (RRRR...GGGG...BBBB...)

On the other hand, with a planar bitstream (RRRR...GGGG...BBBB...), this issue does not occur. In this layout, red pixels are processed in one warp, while green and blue pixels are processed in their respective warps. This approach minimizes thread divergence because the threads within the same warp are working on the same color channel, which means they have similar processing times. The red pixels are processed together, and the green and blue pixels are handled in separate warps, leading to less thread divergence.

Furthermore, the bitstream is first converted into a planar format by the CPU before processing. Afterward, the image is processed, and once the processing is complete, the bitstream is converted back to an interleaved format by the CPU to generate the final image.

The measurements are presented in Table 1.4. These measurements were repeated using a single block and a variable number of threads with the planar bitstream, which will also lead to striding.

Number of threads per block	Number of warps per block	Number of Blocks	Timing [ms]
1	1	1	586,52
2	1	1	299,96
4	1	1	156,36
8	1	1	85,32
16	1	1	48,99
32	1	1	31,56
64	2	1	16,57
128	4	1	8,74
256	8	1	4,70
512	16	1	2,59
1024	32	1	1,55

Table 1.4: Measurements with planar bitstream

In conclusion, the measurement results for the planar bitstream are faster due to reduced thread divergence, as the processing of red, green, and blue pixels is handled more efficiently within separate warps. However, we find the opposite to be true: interleaved is faster than planar for this example. We do not know the reason for this ourselves. The orginal and processed images are shown in figure 1.4 and 1.5.



Figure 1.4: Orginal image



Figure 1.5: Processed image with Gaussian formula

The code for this section can be seen in code snippet 1.2.

CONCLUSION

This lab made us explore GPU optimization techniques for image processing by examining warps, thread starvation, striding, and thread divergence. The results show that efficient GPU utilization depends on the proper allocation of threads and blocks.

Without striding, ensuring enough threads for full image processing is crucial to prevent unprocessed pixels. Implementing striding significantly improves performance by allowing threads to handle multiple data points, reducing execution time. The results confirm that using a multiple of 32 threads per block optimally aligns with the warp size for maximum efficiency.

Thread divergence was analyzed by comparing interleaved and planar bitstreams. The interleaved format led to performance inefficiencies due to mixed RGB processing times within warps, causing idle threads. In contrast, the planar format minimized divergence by grouping pixels of the same color in separate warps, leading to improved execution times.

Overall, this lab showed us the importance of optimizing thread management and memory access patterns for efficient parallel image processing on GPUs.

Listing 1.1: Voorbeeld van CUDA-code

```
/*
 * Code snippet for importing / exporting image data.
 *
 * To convert an image to a pixel map, run 'convert <name>.<extension> <name>.ppm
 *
 */
#include <cstdint>      // Data types
#include <iostream>     // File operations
#include <cuda_runtime.h> // CUDA runtime

// #define M 512        // Lenna width
// #define N 512        // Lenna height
#define M 960        // VR width
#define N 1280        // VR height
#define C 3          // Colors
#define OFFSET 16    // Header length


uint8_t* get_image_array(void){
    /*
     * Get the data of an (RGB) image as a 1D array.
     *
     * Returns: Flattened image array.
     *
     * Notes:
     *  - Images data is flattened per color, column, row.
     *  - The first 3 data elements are the RGB components
     *  - The first 3*M data elements represent the firts row of the image
     *  - For example, r_{0,0}, g_{0,0}, b_{0,0}, ..., b_{0,M}, r_{1,0}, ..., b_{b,M}, ..., b_{N,M}
     *
     */
    // Try opening the file
    FILE *imageFile;
    imageFile=fopen("./anna.ppm","rb");
    if(imageFile==NULL){
        perror("ERROR: Cannot open output file");
        exit(EXIT_FAILURE);
    }

    // Initialize empty image array
    uint8_t* image_array = (uint8_t*)malloc(M*N*C*sizeof(uint8_t)+OFFSET);

    // Read the image
    fread(image_array, sizeof(uint8_t), M*N*C*sizeof(uint8_t)+OFFSET, imageFile);

    // Close the file
    fclose(imageFile);

    // Move the starting pointer and return the flattened image array
    return image_array + OFFSET;
}


void save_image_array(uint8_t* image_array){
    /*
     * Save the data of an (RGB) image as a pixel map.
     *
```

```
58      * Parameters:
59      *   - param1: The data of an (RGB) image as a 1D array
60      *
61      */
62     // Try opening the file
63     FILE *imageFile;
64     imageFile=fopen("./output_image.ppm","wb");
65     if(imageFile==NULL){
66         perror("ERROR: Cannot open output file");
67         exit(EXIT_FAILURE);
68     }
69 
70     // Configure the file
71     fprintf(imageFile,"P6\n");               // P6 filetype
72     fprintf(imageFile,"%d %d\n", M, N);      // dimensions
73     fprintf(imageFile,"255\n");              // Max pixel
74 
75     // Write the image
76     fwrite(image_array, 1, M*N*C, imageFile);
77 
78     // Close the file
79     fclose(imageFile);
80 }
81 
82 
83 
84 __global__ void invert_color(uint8_t* input_array, uint8_t* output_array){
85     int index = threadIdx.x + blockIdx.x * blockDim.x;
86     // Invert the colors
87     if(index < M*N*C){  // Check if the index is within the image size
88         output_array[index] = 255 - input_array[index];
89     }
90 }
91 
92 
93 __global__ void invert_color_with_striding(uint8_t* input_array, uint8_t*
        output_array) {
94     int index = threadIdx.x + blockIdx.x * blockDim.x;
95     int stride = blockDim.x * gridDim.x;  // Stride over het volledige grid
96 
97     // Loop door de afbeelding met stappen van 'stride'
98     for (int i = index; i < M * N * C; i += stride) {
99         output_array[i] = 255 - input_array[i]; // Inverteer kleur
100    }
101 }
102 
103 
104 
105 int main (void) {
106 
107    // Read the image
108    uint8_t* image_array = get_image_array();
109 
110    // Allocate output
111    uint8_t* new_image_array = (uint8_t*)malloc(M*N*C);
112 
113    // Allocate device memory
114    uint8_t* d_input, *d_output;
115    cudaMalloc(&d_input, M*N*C);
116    cudaMalloc(&d_output, M*N*C);
117 
```

```
    // Copy data to device
    cudaMemcpy(d_input, image_array, M*N*C, cudaMemcpyHostToDevice);



    int threadsPerBlock = 1;

    int blocksPerGrid = 1; //fixed number of blocks
    //int blocksPerGrid = ((M*N*C + threadsPerBlock - 1) / threadsPerBlock);    //round up so that all elements are processed
    //int blocksPerGrid = (((M*N*C + threadsPerBlock - 1) / threadsPerBlock) /16) ;    //make sure we have not enough blocks

    for (threadsPerBlock; threadsPerBlock <= 1024; threadsPerBlock*=2){
        invert_color_with_striding<<<blocksPerGrid, threadsPerBlock>>>(d_input, d_output);
        float averageTime = 0;
        // Start Timing GPU
        cudaEvent_t start_cuda, stop_cuda;

        for (size_t i = 0; i < 5; i++)
        {
            cudaEventCreate(&start_cuda);
            cudaEventCreate(&stop_cuda);
            cudaEventRecord(start_cuda);

            invert_color_with_striding<<<blocksPerGrid, threadsPerBlock>>>(d_input, d_output);

            // Stop Timing GPU
            cudaDeviceSynchronize();
            cudaEventRecord(stop_cuda);
            cudaEventSynchronize(stop_cuda);
            float msCuda;
            cudaEventElapsedTime(&msCuda, start_cuda, stop_cuda);
            printf("Time taken: %f ms\n", msCuda);
            averageTime += msCuda;
            cudaEventDestroy(start_cuda);
            cudaEventDestroy(stop_cuda);

        }
        averageTime = averageTime / 5;
        printf("Number of threads per block: %d\n", threadsPerBlock);
        printf("Average time taken: %f ms\n", averageTime);

    }




    cudaMemcpy(new_image_array, d_output, M*N*C, cudaMemcpyDeviceToHost);


    // Save the image
    save_image_array(new_image_array);

    // Free memory
    cudaFree(d_input);
    cudaFree(d_output);
    free(image_array);
```

```
175     free(new_image_array);
176
177     return 0;
178 }
179 }
```

Listing 1.2: Voorbeeld van CUDA-code

```
#include <cstdint>
#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <cuda_runtime.h>

#define M 960
#define N 1280
#define C 3
#define OFFSET 16

uint8_t* get_image_array(void) {
    FILE *imageFile = fopen("./anna.ppm", "rb");
    if (imageFile == NULL) {
        perror("ERROR: Cannot open input file");
        exit(EXIT_FAILURE);
    }
    uint8_t* buffer = (uint8_t*)malloc(M * N * C + OFFSET);
    size_t read_count = fread(buffer, sizeof(uint8_t), M * N * C + OFFSET,
        imageFile);
    if (read_count != M * N * C + OFFSET) {
        perror("ERROR: Could not read the entire image file");
        exit(EXIT_FAILURE);
    }
    fclose(imageFile);
    uint8_t* image_array = (uint8_t*)malloc(M * N * C);
    memcpy(image_array, buffer + OFFSET, M * N * C);
    free(buffer);
    return image_array;
}

void save_image_array(uint8_t* image_array) {
    FILE *imageFile = fopen("./output_image.ppm", "wb");
    if (imageFile == NULL) {
        perror("ERROR: Cannot open output file");
        exit(EXIT_FAILURE);
    }
    fprintf(imageFile, "P6\n%d %d\n255\n", M, N);
    fwrite(image_array, 1, M * N * C, imageFile);
    fclose(imageFile);
}

// Convert an interleaved RGB image to planar format
void convert_rgb_to_planar(uint8_t* input, uint8_t* output) {
    for (int i = 0; i < M * N; i++) {
        output[i] = input[3 * i];
        output[i + M * N] = input[3 * i + 1];
        output[i + 2 * M * N] = input[3 * i + 2];
    }
}

// Convert a planar image to interleaved RGB format
void convert_planar_to_rgb(uint8_t* input, uint8_t* output) {
    for (int i = 0; i < M * N; i++) {
        output[3 * i] = input[i];
        output[3 * i + 1] = input[i + M * N];
        output[3 * i + 2] = input[i + 2 * M * N];
    }
}
```

```
60
61 __global__ void process_image_gaussian_with_striding(uint8_t* input,
       uint8_t* output, bool is_planar) {
62     int index = threadIdx.x + blockIdx.x * blockDim.x;
63     int stride = blockDim.x * gridDim.x;
64     int plane_size = M * N;
65
66     for (int i = index; i < M * N; i += stride) {
67         int red_idx = i;
68         int green_idx = i + plane_size;
69         int blue_idx = i + 2 * plane_size;
70
71         if (is_planar) {
72             // Gaussian on red channel
73             if ((i % M) >= 2 && (i % M) < M - 2) {
74                 output[red_idx] = 0.1f * input[red_idx - 2] + 0.25f * input
                       [red_idx - 1] +
75                                   0.5f * input[red_idx] + 0.25f * input[
                                       red_idx + 1] +
76                                   0.1f * input[red_idx + 2];
77             } else {
78                 output[red_idx] = input[red_idx];
79             }
80
81             // Invert green and blue channels
82             output[green_idx] = 255 - input[green_idx];
83             output[blue_idx] = 255 - input[blue_idx];
84         } else {
85             int rgb_idx = i * 3;
86             if ((i % M) >= 2 && (i % M) < M - 2) {
87                 output[rgb_idx] = 0.1f * input[rgb_idx - 6] + 0.25f * input
                       [rgb_idx - 3] +
88                                   0.5f * input[rgb_idx] + 0.25f * input[
                                       rgb_idx + 3] +
89                                   0.1f * input[rgb_idx + 6];
90             } else {
91                 output[rgb_idx] = input[rgb_idx];
92             }
93             output[rgb_idx + 1] = 255 - input[rgb_idx + 1];
94             output[rgb_idx + 2] = 255 - input[rgb_idx + 2];
95         }
96     }
97 }
98
99
100 int main(void) {
101     uint8_t* image_array = get_image_array();
102     uint8_t* planar_image_array = (uint8_t*)malloc(M * N * C);
103     uint8_t* output_image_array_planar = (uint8_t*)malloc(M * N * C);
104     uint8_t* output_image_array_rgb = (uint8_t*)malloc(M * N * C);
105
106     // Allocate device memory
107     uint8_t *d_input, *d_output;
108     cudaMalloc(&d_input, M * N * C);
109     cudaMalloc(&d_output, M * N * C);
110
111     bool use_planar = true;
112
113     if (use_planar) {
114         convert_rgb_to_planar(image_array, planar_image_array);
115         cudaMemcpy(d_input, planar_image_array, M * N * C,
```

```
            cudaMemcpyHostToDevice);
    } else {
        cudaMemcpy(d_input, image_array, M * N * C, cudaMemcpyHostToDevice)
            ;
    }

    int threadsPerBlock = 1;
    int blocksPerGrid = 1;
    //int blocksPerGrid = (M * N + threadsPerBlock - 1) / threadsPerBlock;

    for (threadsPerBlock; threadsPerBlock <= 1024; threadsPerBlock*=2){
        process_image_gaussian_with_striding<<<blocksPerGrid,
            threadsPerBlock>>>(d_input, d_output, use_planar);
        float averageTime = 0;
        // Start Timing GPU
        cudaEvent_t start_cuda, stop_cuda;

        for (size_t i = 0; i < 5; i++) {
            cudaEventCreate(&start_cuda);
            cudaEventCreate(&stop_cuda);
            cudaEventRecord(start_cuda);

            process_image_gaussian_with_striding<<<blocksPerGrid,
                threadsPerBlock>>>(d_input, d_output, use_planar);

            // Stop Timing GPU
            cudaDeviceSynchronize();
            cudaEventRecord(stop_cuda);
            cudaEventSynchronize(stop_cuda);
            float msCuda;
            cudaEventElapsedTime(&msCuda, start_cuda, stop_cuda);
            printf("Time␣taken:␣%f␣ms\n", msCuda);
            averageTime += msCuda;
            cudaEventDestroy(start_cuda);
            cudaEventDestroy(stop_cuda);

        }
        averageTime = averageTime / 5;
        printf("Number␣of␣threads␣per␣block:␣%d\n", threadsPerBlock);
        printf("Average␣time␣taken:␣%f␣ms\n", averageTime);


    }

    if (use_planar) {
        // Convert the output back to interleaved RGB format
        cudaMemcpy(output_image_array_planar, d_output, M * N * C,
            cudaMemcpyDeviceToHost);
        convert_planar_to_rgb(output_image_array_planar,
            output_image_array_rgb);
        save_image_array(output_image_array_rgb);
    } else {
        cudaMemcpy(output_image_array_rgb, d_output, M * N * C,
            cudaMemcpyDeviceToHost);
        save_image_array(output_image_array_rgb);
    }

    cudaFree(d_input);
    cudaFree(d_output);

    free(image_array);
```

```
170     free(planar_image_array);
171     free(output_image_array_planar);
172     free(output_image_array_rgb);
173     return 0;
174 }
```